

# Sherlock Internship Challenge

## Candidate Identity Fusion Engine

*Phase-by-phase implementation blueprint for a 1.5-day build sprint*

Prepared for: Ritika Gupta | Date: July 9, 2026

**Core submission thesis**

Build the identity routing layer Sherlock needs before fraud detectors can run safely: a real-time system that fuses metadata, participant events, transcript evidence, and behavior signals to identify the candidate with confidence, explanation, and graceful uncertainty.

| Decision          | Final choice                                                                                          |
|-------------------|-------------------------------------------------------------------------------------------------------|
| Repo architecture | Turborepo monorepo with apps/api, apps/web, packages/core, packages/db, packages/shared, packages/llm |
| Backend           | Node.js + TypeScript + Fastify + @fastify/websocket                                                   |
| Frontend          | React + Vite + TypeScript dashboard                                                                   |
| Database          | Postgres + Prisma, with in-memory fallback for tests                                                  |
| Validation        | Zod schemas shared across API, evaluator, and frontend                                                |
| LLM use           | Structured transcript role evidence only; rules/fusion engine remains final decision-maker            |
| Testing           | Vitest unit/integration tests + scenario evaluation harness                                           |
| Demo path         | Replay JSON meeting scenarios over WebSocket and show live confidence timeline                        |

Primary source alignment: the challenge asks for automatic real-time candidate identification, confidence updates, incorrect/missing names handling, ambiguity handling, explainability, a demo, architecture diagram, evaluation, edge cases, accuracy, and limitations.

# Contents

- 1. What we are solving
- 2. Final architecture
- 3. Repository structure
- 4. Data model and event contracts
- 5. Scoring, confidence, and state machine
- 6. Phase-by-phase implementation plan
- 7. Evaluation harness and edge-case suite
- 8. 1.5-day execution schedule
- 9. Demo video and README checklist
- 10. Reference materials
- 11. AI concepts integration map

## 1. What we are solving

The internship challenge is not asking us to build Sherlock's full anti-fraud system. It asks us to identify which participant is the interview candidate during a live meeting. Sherlock already has or discusses fraud detectors such as deepfake detection, voice cloning, and behavioral analysis; the challenge is that those detectors must be routed to the correct participant stream.

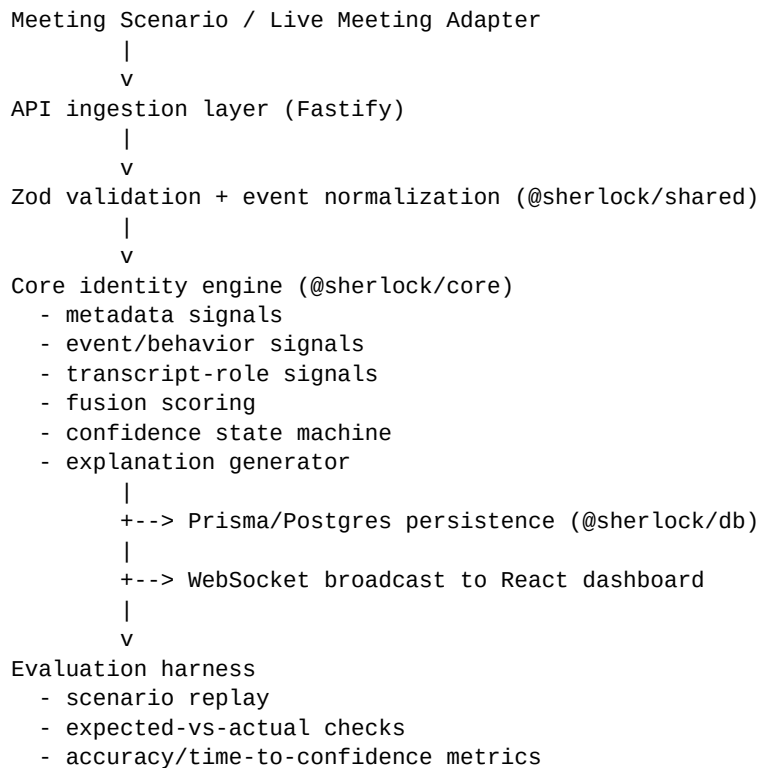
The strongest prototype should therefore behave like a real-time identity resolution layer: it should track every participant, update evidence over time, rank participants, provide a confidence score, show the second-best candidate, and refuse to force a decision when evidence is weak or ambiguous.

**Non-goals for the sprint**  
Do not build full Zoom/Meet integration, full deepfake detection, voice biometrics, face recognition, or a heavy multi-agent framework. These can be mentioned as future extensions. The 1.5-day demo should prove identity fusion, explainability, and edge-case evaluation.

| Challenge requirement                | Implementation response                                                                                                                          |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Automatically identify the candidate | Fusion engine ranks all participants and selects top candidate only above threshold.                                                             |
| Continuously update confidence       | Every incoming event triggers score recomputation and WebSocket broadcast.                                                                       |
| Handle incorrect names               | Name matching is only one weak signal; transcript/behavior can override weak metadata.                                                           |
| Handle missing information           | State remains INSUFFICIENT_DATA until enough evidence appears.                                                                                   |
| Handle ambiguous situations          | Margin rule returns AMBIGUOUS when top two participants are close.                                                                               |
| Explain why selected                 | Every signal emits evidence items with weight, impact, and human-readable reason.                                                                |
| Evaluation                           | Scenario harness runs edge-case JSON files and reports top-1 accuracy, ambiguity handling, time-to-confidence, and false interviewer selections. |

## 2. Final architecture

The architecture is deliberately split into pure, testable packages and thin delivery layers. The core identity engine must not depend on Fastify, WebSocket, React, Prisma, or an LLM provider. That keeps the most important logic testable with Vitest and reusable in a production Sherlock service later.



| Layer           | Responsibility                                                                            | Why this helps                                                              |
|-----------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| packages/shared | Zod schemas, event types, API message contracts.                                          | One contract across API, evaluator, and frontend. Reduces schema drift.     |
| packages/core   | Pure scoring, signal extraction, state transitions, explanation generation.               | Can be tested fast without DB/API/frontend. This is the engineering center. |
| packages/db     | Prisma schema, migrations, typed DB client, seed helpers.                                 | Provides persistence without leaking DB into core logic.                    |
| packages/llm    | Transcript classifier adapter with provider interface and fallback rules.                 | LLM adds evidence, but cannot directly decide the candidate.                |
| apps/api        | Fastify REST + WebSocket ingestion/broadcast API.                                         | Production-style real-time backend with thin orchestration.                 |
| apps/web        | React dashboard for scenario replay, live rankings, explanations, and evaluation results. | Demo-ready UI that shows reasoning, not only final answer.                  |
| scenarios       | JSON meeting simulations and expected outcomes.                                           | Repeatable evaluation and edge-case proof.                                  |

### 3. Repository structure

```
sherlock-candidate-identity/
  apps/
    api/
      src/
        index.ts
        routes/
          meetings.ts
          scenarios.ts
        realtime/
          wsServer.ts
          broadcaster.ts
        services/
          meetingService.ts
          scenarioReplayService.ts
```

```

package.json
web/
src/
  App.tsx
  api/client.ts
  ws/useCandidateSocket.ts
  components/
    CandidateCard.tsx
    ParticipantLeaderboard.tsx
    EvidencePanel.tsx
    EventTimeline.tsx
    EvaluationSummary.tsx
package.json
packages/
shared/
src/
  schemas.ts
  types.ts
  websocket.ts
package.json
core/
src/
  signals/
    metadataSignals.ts
    behaviorSignals.ts
    transcriptSignals.ts
  fusion/
    fusionEngine.ts
    confidence.ts
    stateMachine.ts
  explanation/
    explanationEngine.ts
  scenario/
    scenarioRunner.ts
  evaluation/
    evaluator.ts
  tests/
package.json
db/
  prisma/schema.prisma
  src/client.ts
  src/repositories.ts
package.json
llm/
  src/provider.ts
  src/openaiTranscriptClassifier.ts
  src/ruleFallbackClassifier.ts
package.json
scenarios/
  01_exact_name_match.json
  02_candidate_as_macbook_pro.json
  03_wrong_candidate_name.json
  04_multiple_interviewers.json
  05_silent_observer.json
  06_display_name_change.json
  07_candidate_late_join.json
  08_two_unknown_participants_ambiguous.json
  09_candidate_rejoins.json
  10_no_transcript_yet.json
docker-compose.yml
package.json
bun.lock

```

## 4. Data model and event contracts

The data contract should reflect the challenge inputs: participant information, join/leave events, webcam/screen-share status, speaking activity, speaker-attributed transcript, and external metadata such as candidate name/email, calendar invite, interview schedule, and interviewer names.

| Entity         | Key fields                                                                                             |
|----------------|--------------------------------------------------------------------------------------------------------|
| Meeting        | id, candidateName, candidateEmail, scheduledStart, interviewerNames, interviewerEmails, companyDomains |
| Participant    | id, meetingId, displayName, email, currentName, joinedAt, leftAt, isKnownInterviewerHint               |
| MeetingEvent   | id, meetingId, participantId, type, timestamp, payload                                                 |
| ScoreSnapshot  | meetingId, participantId, confidence, rawScore, state, evidence[], uncertainty[], createdAt            |
| ScenarioResult | scenarioId, expectedCandidateId, predictedCandidateId, passed, metrics, finalSnapshot                  |

```
// Core event examples
export type MeetingEvent =
  | { type: 'participant_joined'; participantId: string; timestampSec: number }
  | { type: 'participant_left'; participantId: string; timestampSec: number }
  | { type: 'display_name_changed'; participantId: string; timestampSec: number; newDisplayName:
string }
  | { type: 'webcam_changed'; participantId: string; timestampSec: number; webcamOn: boolean }
  | { type: 'screen_share_changed'; participantId: string; timestampSec: number; sharing:
boolean }
  | { type: 'speaking_activity'; participantId: string; timestampSec: number; durationSec:
number }
  | { type: 'transcript_chunk'; participantId: string; timestampSec: number; text: string };
// WebSocket message shape emitted to React
{
  "type": "candidate_state_updated",
  "meetingId": "meet_001",
  "selectedCandidateId": "p2",
  "confidence": 0.84,
  "state": "LIKELY_CANDIDATE",
  "participants": [
    { "participantId": "p2", "displayName": "MacBook Pro", "confidence": 0.84 },
    { "participantId": "p1", "displayName": "Priya Sharma", "confidence": 0.19 }
  ],
  "evidence": [
    { "signal": "transcript_role", "impact": 0.28, "reason": "Speaker introduced their own
project." }
  ],
  "uncertainty": ["Display name did not match candidate name."]
}
```

## 5. Scoring, confidence, and state machine

The first implementation should use weighted evidence, not a trained model. We do not have real labeled meeting data, and the challenge values explainability. Each signal returns a normalized score contribution and a reason. Later, weights can be learned from labeled historical meetings.

| Signal group | Example signals | Default weight |
|--------------|-----------------|----------------|
|--------------|-----------------|----------------|

|                           |                                                                                                |      |
|---------------------------|------------------------------------------------------------------------------------------------|------|
| Metadata                  | candidate name/email similarity, interviewer exclusion, company domain penalty                 | 0.30 |
| Transcript role           | self-introduction, candidate-answer language, interviewer-question language, observer language | 0.30 |
| Behavior/audio            | speaking duration, answer-after-question pattern, silent observer penalty                      | 0.18 |
| Meeting events            | join timing, display-name continuity, rejoin continuity                                        | 0.12 |
| Video/screen weak signals | webcam on, screen share during technical answer                                                | 0.10 |

```
rawScore(participant) =  
  0.30 * metadataScore +  
  0.30 * transcriptRoleScore +  
  0.18 * behaviorScore +  
  0.12 * eventScore +  
  0.10 * weakVideoScreenScore
```

```
confidence(participant) = softmax(rawScores)[participant]
```

```
Decision rules:  
- topConfidence < 0.55 -> INSUFFICIENT_DATA  
- topConfidence >= 0.55 and < 0.75 -> POSSIBLE_CANDIDATE  
- topConfidence >= 0.75 and margin >= 0.15 -> LIKELY_CANDIDATE  
- topConfidence >= 0.90 and stable >= 60 sec -> CONFIRMED_CANDIDATE  
- topConfidence - secondConfidence < 0.15 -> AMBIGUOUS
```

**Important design principle**

The LLM never chooses the candidate directly. It only classifies transcript chunks into structured evidence: candidate-like, interviewer-like, observer-like, or unknown. The deterministic fusion engine makes the final decision.

## 6. Phase-by-phase implementation plan

Each phase has an architecture goal, implementation output, tests, evaluation gate, and reference material. Do not move to the next phase until the gate passes. This makes the project feel like an engineering system, not a one-off demo.

### Phase 0 — Product scope and acceptance criteria

Objective: Lock the exact problem and define what the demo must prove.

Architecture focus: No code yet. Convert the PDF challenge into acceptance tests: identify candidate, update confidence, handle missing/incorrect names, ambiguity, explanations, edge cases.

Implementation tasks:

- Create README.md with problem statement, non-goals, and thesis.
- Create ARCHITECTURE.md with first Mermaid diagram.
- Write a scenario list before writing code.

Tests and evaluation gate:

- Manual review: each challenge requirement maps to one feature and at least one scenario.

Commands:

No commands yet.

### Definition of done

README has the one-sentence thesis and a requirement-to-feature table.

## Phase 1 — Initialize Turborepo and workspace contracts

Objective: Set up a monorepo where API, web, core, db, shared, and llm packages can evolve independently.

Architecture focus: Turborepo manages task graphs; Bun workspaces manage package relationships. packages/shared becomes the source of truth for Zod schemas and TypeScript types.

Implementation tasks:

- Initialize repo with Bun and Turborepo.
- Create apps/api, apps/web, packages/shared, packages/core, packages/db, packages/llm.
- Add TypeScript, tsconfig base, ESLint/Prettier optional.
- Create shared Zod schemas for Meeting, Participant, MeetingEvent, EvidenceItem, CandidateState.

Tests and evaluation gate:

- Bun/Turborepo typecheck succeeds across workspace.
- Vitest smoke test imports shared schemas from packages/core.
- Zod rejects malformed events.

Commands:

```
bun install
bunx turbo run typecheck
bun run test -- --run
```

### Definition of done

A malformed transcript event fails validation; a valid event passes and is imported by core tests.

## Phase 2 — Write the DB package with Prisma/Postgres

Objective: Persist meetings, participants, events, score snapshots, and scenario results without coupling DB code to the core engine.

Architecture focus: packages/db owns Prisma schema and repositories. apps/api calls repositories. packages/core remains pure and accepts plain objects.

Implementation tasks:

- Add docker-compose.yml for Postgres.
- Add Prisma schema for Meeting, Participant, MeetingEvent, ScoreSnapshot, ScenarioResult.
- Create Prisma client export and repository functions.
- Add seed script for one demo meeting.

Tests and evaluation gate:

- Repository integration test can create a meeting, participants, events, and score snapshot.
- DB tests run against local Postgres or a test database URL.
- Core package does not import Prisma.

Commands:

```
docker compose up -d
bun --filter @sherlock/db run prisma:migrate
bun --filter @sherlock/db run test
```

#### Definition of done

One scenario can be stored and reloaded from Postgres with all event payloads intact.

## Phase 3 — Build core domain model and deterministic signal extractors

Objective: Build the first real candidate identification logic before WebSocket or frontend.

Architecture focus: packages/core contains pure functions: `applyEvent(sessionState, event)`, `extractSignals(sessionState)`, `rankParticipants(sessionState)`.

Implementation tasks:

- Implement name/email similarity using rapidfuzz-equivalent package or simple normalized similarity.
- Implement interviewer exclusion and company-domain penalty.
- Implement join timing, display-name change continuity, webcam/screen-share weak signals.
- Implement speaking duration and silent observer penalty.
- Each signal returns `EvidenceItem[]` and `uncertainty[]` instead of only numbers.

Tests and evaluation gate:

- Unit tests for exact name, nickname, MacBook Pro, known interviewer, silent observer.
- Every signal test asserts both score and explanation text.
- No API/DB/frontend required to run tests.

Commands:

```
bun --filter @sherlock/core run test -- signals
```

#### Definition of done

Core can rank participants in simple metadata/event cases and produce evidence items.

## Phase 4 — Fusion engine, confidence, and state machine

Objective: Turn signal outputs into a stable candidate decision with confidence and ambiguity handling.

Architecture focus: `fusionEngine.ts` computes raw scores and softmax confidence. `stateMachine.ts` maps confidence and margin into `INSUFFICIENT_DATA`, `POSSIBLE_CANDIDATE`, `LIKELY_CANDIDATE`, `CONFIRMED_CANDIDATE`, or `AMBIGUOUS`.

Implementation tasks:

- Implement weighted scoring and confidence normalization.
- Track top candidate and second-best participant.
- Implement margin rule for ambiguity.
- Implement stability timer for `CONFIRMED_CANDIDATE`.
- Implement explanation aggregation sorted by absolute impact.

Tests and evaluation gate:

- Correct candidate with strong evidence becomes `LIKELY_CANDIDATE`.
- Two close candidates becomes `AMBIGUOUS`.
- No transcript yet remains `POSSIBLE` or `INSUFFICIENT_DATA` depending metadata.

- Known interviewer is never selected unless all data is contradictory and marked ambiguous.

Commands:

```
bun --filter @sherlock/core run test -- fusion state
```

#### Definition of done

Core returns selectedCandidateId, confidence, state, participant rankings, evidence, uncertainty.

## Phase 5 — Scenario simulator and evaluation harness

Objective: Make the submission measurable and repeatable. This is the part that will make the project stand out.

Architecture focus: Scenario runner replays JSON events through the pure core engine. Evaluator compares predicted candidate and state with expected output after each scenario.

Implementation tasks:

- Create 10 scenario JSON files covering challenge edge cases.
- Implement scenarioRunner.ts to apply events one by one.
- Implement evaluator.ts with metrics: top-1 accuracy, top-2 accuracy, false interviewer selection, ambiguity precision, time-to-likely-candidate.
- Save evaluation report as JSON and print a readable CLI summary.

Tests and evaluation gate:

- End-to-end scenario tests pass for at least 8/10 initial scenarios.
- Ambiguous scenario should not be forced into a candidate selection.
- MacBook Pro scenario should improve after self-introduction transcript arrives.

Commands:

```
bun run evaluate
bun --filter @sherlock/core run test -- scenario
```

#### Definition of done

CLI prints scenario pass/fail table and aggregate metrics.

## Phase 6 — WebSocket layer and API ingestion

Objective: Expose the engine as a real-time service. This proves the system can continuously update confidence during the interview.

Architecture focus: apps/api receives events over REST or scenario replay, calls core engine, persists events/snapshots through db package, and broadcasts candidate\_state\_updated over WebSocket.

Implementation tasks:

- Create Fastify server with health endpoint.
- Register @fastify/websocket before normal routes.
- Add /meetings/:id/events POST to ingest one event.
- Add /scenarios/:id/replay endpoint to stream scenario events.
- Broadcast score snapshots to connected dashboard clients.

Tests and evaluation gate:

- API health check returns ok.
- Posting event produces a score snapshot.
- WebSocket test client receives candidate\_state\_updated.
- Scenario replay sends multiple updates over WebSocket.

Commands:

```
bun --filter api run dev
bun --filter api run test
```

#### Definition of done

A test WebSocket client receives live confidence updates for a replayed scenario.

## Phase 7 — LLM transcript classifier package

Objective: Use AI where it helps most: interpreting transcript role evidence while keeping the final decision deterministic and auditable.

Architecture focus: packages/llm exposes TranscriptClassifier interface. Implement OpenAI/Gemini adapter with structured JSON output and a deterministic ruleFallbackClassifier for offline tests.

Implementation tasks:

- Define TranscriptRoleEvidence schema using Zod.
- Implement rule-based fallback classifier for no-API mode.
- Implement provider adapter that asks for roleEvidence: candidate/interviewer/observer/unknown, confidence, reason, extractedIdentityHints.
- Cache LLM classification by transcript text hash to avoid repeated calls.
- Add env flag USE\_LLM=true/false.

Tests and evaluation gate:

- Offline tests run with rule fallback only.
- Mock LLM tests validate structured output parsing.
- Invalid LLM output is rejected and falls back safely.
- Transcript evidence changes candidate score but never bypasses fusion thresholds.

Commands:

```
bun --filter @sherlock/llm run test
USE_LLM=false bun run evaluate
USE_LLM=true bun run evaluate
```

#### Definition of done

Evaluation works without paid APIs; with API key, transcript evidence becomes richer.

## Phase 8 — React real-time dashboard

Objective: Make the demo visually understandable: show how the system reaches the conclusion, not just who it selected.

Architecture focus: apps/web connects to WebSocket, selects scenarios, starts replay, and renders current candidate, confidence timeline, participant leaderboard, evidence cards, uncertainty warnings, and evaluation summary.

Implementation tasks:

- Build scenario selector and Start Replay button.

- Build CandidateCard showing selected candidate, state, confidence, and top competing participant.
- Build ParticipantLeaderboard showing all participant confidence values.
- Build EvidencePanel showing positive/negative signals and uncertainty.
- Build EventTimeline showing raw incoming events.

Tests and evaluation gate:

- Component tests for rendering candidate states.
- Manual demo test: replay MacBook Pro scenario and watch confidence increase.
- Manual ambiguity test: dashboard displays AMBIGUOUS with second-best close to top.

Commands:

```
bun --filter web run dev
bun --filter web run test
```

#### Definition of done

Dashboard can replay at least 3 scenarios live and make decisions explainable to a reviewer.

## Phase 9 — Edge-case hardening and evaluation report

Objective: Make the system robust and honest. The evaluator should prove behavior across the challenge's tricky cases.

Architecture focus: Evaluation becomes a first-class deliverable: scenario files, metrics, failure notes, and limitations are included in README.

Implementation tasks:

- Add edge cases: wrong candidate name, multiple interviewers, silent observer, display-name change, late join, candidate rejoin, two unknown participants, no transcript yet.
- Tune thresholds only through scenario evaluation, not by eyeballing one demo.
- Generate evaluation report JSON and copy summary into README.
- Add limitations section: no real Zoom integration, no enrolled face/voice biometrics, synthetic scenarios.

Tests and evaluation gate:

- Run full evaluator and record metrics.
- At least one intentionally ambiguous scenario should be correctly ambiguous.
- False interviewer selections should be zero in the provided scenarios.

Commands:

```
bun run evaluate > evaluation_report.txt
bun run test -- --run
```

#### Definition of done

README includes exact metrics, edge-case table, and limitations.

## Phase 10 — Submission polish: README, architecture diagram, video script

Objective: Package the project so reviewers understand the thinking quickly.

Architecture focus: No major new code. Polish documentation and record a 5–10 minute demo that walks through architecture, approach, demo, trade-offs, and improvements.

Implementation tasks:

- Add setup instructions with env vars and commands.
- Add architecture diagram and explanation of each package.
- Add demo scenarios section with screenshots or GIF optional.
- Add evaluation results and limitations.
- Prepare video script: problem, architecture, demo, evaluation, tradeoffs, future work.

Tests and evaluation gate:

- Fresh clone setup test if possible.
- Run bun install, docker compose up, migrations, tests, evaluate, dev servers.
- Check no API keys committed.

Commands:

```
bun install
docker compose up -d
bun run db:migrate
bun run test -- --run
bun run evaluate
bun run dev
```

**Definition of done**  
GitHub repository is clean, reproducible, and has a clear reviewer path.

## 7. Evaluation harness and edge-case suite

The evaluation harness should be treated as part of the product. It proves that the system behaves correctly across the exact edge cases listed in the challenge, and it lets us tune weights without breaking earlier scenarios.

| Metric                                | Definition                                                        | Why it matters                                                    |
|---------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------|
| Top-1 accuracy                        | predictedCandidateId == expectedCandidateId                       | Main correctness metric.                                          |
| Top-2 accuracy                        | expected candidate appears in top two participants                | Useful when ambiguity is high but ranking is still useful.        |
| False interviewer selection           | Known interviewer selected as candidate                           | Critical trust failure; should be zero in demo scenarios.         |
| Ambiguity precision                   | Ambiguous scenarios produce AMBIGUOUS instead of a forced guess   | Shows graceful uncertainty.                                       |
| Time to likely candidate              | First timestamp when correct participant reaches LIKELY_CANDIDATE | Proves real-time usefulness.                                      |
| Explanation coverage                  | Every selected state has at least N evidence reasons              | Proves auditability and reviewer trust.                           |
| Scenario                              | Purpose                                                           | Expected behavior                                                 |
| 01_exact_name_match                   | Baseline                                                          | Candidate selected quickly with high metadata confidence.         |
| 02_candidate_as_macbook_pro           | Display name failure                                              | Starts uncertain; transcript self-intro increases confidence.     |
| 03_wrong_candidate_name               | Bad metadata                                                      | Does not over-trust calendar; relies on role behavior/transcript. |
| 04_multiple_interviewers              | Many non-candidates                                               | Known interviewers downweighted; candidate chosen.                |
| 05_silent_observer                    | Observer present                                                  | Silent observer remains low confidence.                           |
| 06_display_name_change                | Name changes mid-call                                             | Participant ID continuity preserves identity.                     |
| 07_candidate_late_join                | Candidate absent initially                                        | System avoids guessing too early.                                 |
| 08_two_unknown_participants_ambiguous | Weak/conflicting evidence                                         | State becomes AMBIGUOUS.                                          |
| 09_candidate_rejoins                  | Leave/rejoin behavior                                             | System re-evaluates but retains continuity                        |

|                      |                     |                                                                              |
|----------------------|---------------------|------------------------------------------------------------------------------|
|                      |                     | when participant ID or identity hints align.                                 |
| 10_no_transcript_yet | Early meeting state | Uses metadata/events but stays lower confidence if evidence is insufficient. |

Expected CLI output shape:

Sherlock Candidate Identity Evaluation

-----

Total scenarios: 10  
Passed: 8  
Failed: 2  
Top-1 accuracy: 80%  
Top-2 accuracy: 100%  
False interviewer selections: 0  
Ambiguity cases handled: 2/2  
Average time to LIKELY\_CANDIDATE: 42s

Scenario Results:

- ✔ 01\_exact\_name\_match
- ✔ 02\_candidate\_as\_macbook\_pro
- ✔ 03\_wrong\_candidate\_name
- ✔ 04\_multiple\_interviewers
- ✔ 05\_silent\_observer
- ✔ 06\_display\_name\_change
- ✔ 07\_candidate\_late\_join
- ✔ 08\_two\_unknown\_participants\_ambiguous
- ✔ 09\_candidate\_rejoins
- ⚠ 10\_no\_transcript\_yet: expected POSSIBLE\_CANDIDATE, got INSUFFICIENT\_DATA

8. 1.5-day execution schedule

This schedule assumes speed matters. The highest-value deliverables are: core engine, scenario evaluator, WebSocket live updates, and dashboard explanations. LLM integration is valuable but should not block the deterministic demo.

| Time block | Focus            | Must finish                                                     |
|------------|------------------|-----------------------------------------------------------------|
| Hour 0-1   | Phase 0-1        | Repo initialized, packages created, shared schemas compile.     |
| Hour 1-3   | Phase 2          | Prisma schema, docker compose Postgres, basic repositories.     |
| Hour 3-6   | Phase 3-4        | Core signals, fusion engine, state machine, unit tests.         |
| Hour 6-8   | Phase 5          | Scenario runner, 6 scenarios minimum, evaluator CLI.            |
| Hour 8-10  | Phase 6          | Fastify ingestion + WebSocket broadcast.                        |
| Hour 10-13 | Phase 8          | React dashboard with scenario replay and evidence panels.       |
| Hour 13-15 | Phase 7 optional | LLM structured transcript classifier adapter + fallback.        |
| Hour 15-18 | Phase 9          | Edge-case hardening, tune thresholds, evaluation report.        |
| Hour 18-22 | Phase 10         | README, architecture diagram, demo video script, final cleanup. |

Priority rule under time pressure

If time becomes tight, skip LLM provider integration before skipping evaluation or dashboard explainability. The submission wins by proving identity reasoning under edge cases, not by calling an API.

## 9. Demo video and README checklist

| README section | What to include                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------|
| Problem        | Why candidate identity routing is required before Sherlock fraud detectors run.                            |
| Approach       | Multiple weak signals + fusion confidence + ambiguity state.                                               |
| Architecture   | Turborepo packages, API, DB, WebSocket, dashboard diagram.                                                 |
| Setup          | bun install, docker compose up, prisma migrate, bun run dev.                                               |
| Demo scenarios | List the JSON scenarios and what each proves.                                                              |
| Evaluation     | Metrics table from bun run evaluate.                                                                       |
| Trade-offs     | Why no full Zoom integration, no face recognition, no voice biometrics in 1.5 days.                        |
| Future work    | Real meeting adapters, learned weights, face/voice enrollment, reviewer feedback loop, production scaling. |

Suggested 5–10 minute video flow:

- 0:00–0:45 Problem framing: Sherlock needs to know which participant is the candidate before fraud detectors can be routed.
- 0:45–2:00 Architecture: monorepo, core engine, DB, WebSocket, React dashboard, evaluation harness.
- 2:00–4:30 Live demo: MacBook Pro candidate scenario; show confidence increasing after transcript evidence.
- 4:30–6:00 Edge cases: wrong candidate name, multiple interviewers, ambiguous participants.
- 6:00–7:30 Evaluation: show bun run evaluate metrics and scenario pass/fail.
- 7:30–9:00 Trade-offs and future work.

## 10. Reference materials

Use these references while implementing or improving the architecture. The first item is the uploaded challenge PDF; the rest are official product/tooling references or company research sources checked for this blueprint.

- **Sherlock Internship Challenge PDF:** Uploaded challenge document. Key requirements: real-time candidate identification, confidence updates, ambiguity handling, explainability, evaluation, edge cases.
- **Sherlock AI website:** <https://www.withsherlock.ai/> — product positioning around real-time interview fraud detection, AI fluency, and note taking.
- **Sherlock AI security overview:** <https://www.withsherlock.ai/blog/sherlock-ai-anti-interview-fraud-security-overview> — data minimization, auditability, identity/authorship verification, fairness.
- **Sherlock AI vs traditional monitoring:** <https://www.withsherlock.ai/blog/sherlock-ai-vs-traditional-interview-monitoring> — real-time interview fraud and continuous verification framing.
- Bun workspaces documentation: <https://bun.com/docs/pm/workspaces> — workspace support for Bun package management in monorepos.
- **Turborepo docs:** <https://turborepo.dev/docs> — build system for JavaScript/TypeScript monorepos.
- **Turborepo package/task graphs:** <https://turborepo.dev/docs/core-concepts/package-and-task-graph> — why internal packages and task relationships matter.
- **Prisma PostgreSQL quickstart:** <https://www.prisma.io/docs/prisma-orm/quickstart/postgresql> — TypeScript + Postgres + Prisma setup.
- **Prisma ORM overview:** <https://www.prisma.io/docs/orm> — type-safe database access and migrations.
- **Fastify WebSocket plugin:** <https://github.com/fastify/fastify-websocket> — WebSocket routes and registration guidance.
- **Vite guide:** <https://vite.dev/guide/> — React/TypeScript frontend setup.
- **Zod documentation:** <https://zod.dev/> — TypeScript-first schema validation.
- **Vitest documentation:** <https://vitest.dev/guide/> — Vite-native testing for frontend and backend TypeScript.

- **OpenAI Structured Outputs:** <https://developers.openai.com/api/docs/guides/structured-outputs> — JSON-schema-constrained model outputs for reliable transcript evidence.
- **Socket.IO documentation:** <https://socket.io/> — alternative real-time layer if room/broadcast scaling becomes more important than raw WebSocket simplicity.

## 11. AI concepts integration map

**Answer to the implementation question:** the plan includes the core AI integrations required for the challenge: evaluation harness, structured LLM outputs/tool calling, weak-signal fusion, and real-time session memory. Agents and RAG are included as lightweight/optional extensions. Fine-tuning and RLM/RL are intentionally not build-phase integrations for the 1.5-day prototype; they belong in the future-improvements section because they require real labeled Sherlock interview data or advanced research infrastructure.

| AI concept                        | Included in plan? | Where it appears                                                     | Implementation depth                                                          |
|-----------------------------------|-------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Evaluation harness                | Yes — core        | Phase 5, Phase 9, evaluation CLI, scenario JSON suite                | Deep implementation. Required to prove behavior across edge cases.            |
| Structured outputs / tool calling | Yes — core        | Phase 7 LLM transcript classifier package                            | LLM returns strict JSON evidence using Zod schema validation.                 |
| Weak-signal fusion                | Yes — core        | Phase 3 metadata/behavior signals and Phase 4 fusion engine          | Main identity brain: combines deterministic and AI-generated evidence.        |
| Session memory                    | Yes — core        | DB package, participant state, confidence history, evidence log      | Event-sourced meeting memory: identity persists through name changes/rejoins. |
| Agents                            | Lightweight       | Optional orchestration layer around signal tools                     | Use an Identity Reasoning Agent trace; avoid heavy multi-agent frameworks.    |
| RAG                               | Optional bonus    | Future Phase 11B if candidate resume/JD/application context is added | Retrieve candidate context to strengthen identity clues from transcript.      |
| Fine-tuning                       | No for MVP        | Future work only                                                     | Needs labeled Sherlock meetings; mention as production improvement.           |
| RLM/RL                            | No for MVP        | Not included                                                         | Unnecessary for this prototype; event-driven identity scoring is enough.      |

### 11.1 Core AI build layers

- A. LLM evidence extractor
  - Input: speaker-attributed transcript chunk plus meeting metadata.
  - Output: strict JSON with speakerRole, confidence, identity clues, and reason.
  - Purpose: understand messy human language such as self-introductions, nicknames, device-name explanations, and candidate-style answers.
  - Test gate: transcript classifier unit tests plus scenario-level accuracy checks.
- B. Deterministic signal tools
  - Input: participant metadata and meeting events.
  - Output: evidence objects for name match, email match, interviewer exclusion, join timing, screen share, webcam state, and speaking duration.
  - Purpose: keep predictable evidence outside the LLM so the system remains explainable and testable.
  - Test gate: per-signal unit tests with expected score ranges.
- C. Fusion and confidence engine

- Input: deterministic evidence plus LLM evidence.
  - Output: ranked participants, selected candidate, confidence, state, second-best margin, uncertainty warnings.
  - Purpose: make the final decision using a transparent scoring policy rather than a black-box LLM guess.
  - Test gate: top-1 accuracy, top-2 accuracy, false interviewer selection, ambiguity handling, time-to-confidence.
- D. Session memory and evidence history
    - Input: all normalized meeting events and score updates.
    - Output: participant timeline, display-name history, confidence history, evidence log, and state transitions.
    - Purpose: maintain continuity when a candidate changes display name, leaves/rejoins, or starts as unknown.
    - Test gate: display-name-change and rejoin scenarios should preserve participant identity correctly.
- E. Explanation generator
    - Input: selected participant, confidence, top evidence, uncertainty reasons.
    - Output: short human-readable explanation for the dashboard and evaluation report.
    - Purpose: satisfy the challenge requirement to explain why the system selected a participant.
    - Test gate: every final decision must include positive evidence, negative evidence, and uncertainty when relevant.

## 11.2 Updated phase alignment

| Phase              | Integration                    | What to build/test                                                                                              |
|--------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Phase 3            | Deterministic signal tools     | Rules-based metadata and behavior evidence.                                                                     |
| Phase 4            | Fusion engine                  | Confidence scoring, state machine, ambiguity handling.                                                          |
| Phase 5            | Evaluation harness             | Scenario replay, metrics, expected-vs-actual checks.                                                            |
| Phase 6            | WebSocket layer                | Real-time event ingestion and live candidate-state broadcast.                                                   |
| Phase 7            | LLM structured evidence        | Transcript role classifier and identity clue extractor using JSON schema output.                                |
| Phase 8            | Dashboard                      | Evidence cards, confidence timeline, participant ranking.                                                       |
| Phase 9            | Edge-case hardening            | Run all edge cases and improve thresholds/explanations.                                                         |
| Optional Phase 11A | Identity Reasoning Agent trace | Show orchestration steps: metadata tool -> transcript tool -> behavior tool -> fusion tool -> explanation tool. |
| Optional Phase 11B | RAG context                    | Add resume/JD/application retrieval only if useful and time remains.                                            |

## 11.3 Optional agent layer without over-engineering

**Use this only if the core engine and evaluation harness are already passing.** The agent should not independently decide the candidate. It should only orchestrate tools and produce an execution trace.

IdentityReasoningAgent

1. runMetadataTool(participants, calendar)
  2. runTranscriptTool(transcriptChunk)
  3. runBehaviorTool(events)
  4. runFusionTool(allEvidence)
  5. runExplanationTool(selection)
- Why include it: it makes the demo feel AI-native and shows tool orchestration.
  - Why keep it lightweight: heavy autonomous agents introduce nondeterminism and are harder to evaluate.

- Evaluation gate: agent trace must be reproducible for each scenario and must not bypass the fusion engine.

## 11.4 Optional RAG layer

**RAG is optional because the challenge already provides meeting metadata and transcript.** Add it only if we want the system to use a candidate resume, job description, or application form as context.

- Example: candidate resume mentions ClaimFlow AI; participant p2 says “I built ClaimFlow AI”; transcript evidence becomes stronger.
- Implementation shortcut: use a small local JSON knowledge base first rather than a vector database.
- Evaluation gate: RAG must improve identity confidence in resume-context scenarios without creating false positives.

## 11.5 Fine-tuning and RLM/RL decision

- Do not fine-tune in the prototype. We do not have real labeled meeting data.
- Mention fine-tuning as a production roadmap: Sherlock could learn signal weights and transcript classifiers from labeled historical meetings.
- Do not include RLM/RL. The challenge is better solved with event-driven identity scoring, structured LLM evidence, and evaluation harnesses.

## 11.6 Final AI positioning for the README/video

This is an AI-assisted real-time candidate identity fusion system.

It uses LLMs where rules are weak: transcript role understanding, identity clue extraction, and explanation generation.

It uses deterministic tools where reliability matters: metadata matching, interviewer exclusion, event handling, confidence thresholds, and evaluation.

The final candidate decision is made by a transparent fusion engine, not a black-box LLM.